

DOSSIER D'ANALYSE TECHNIQUE

BOX'TY

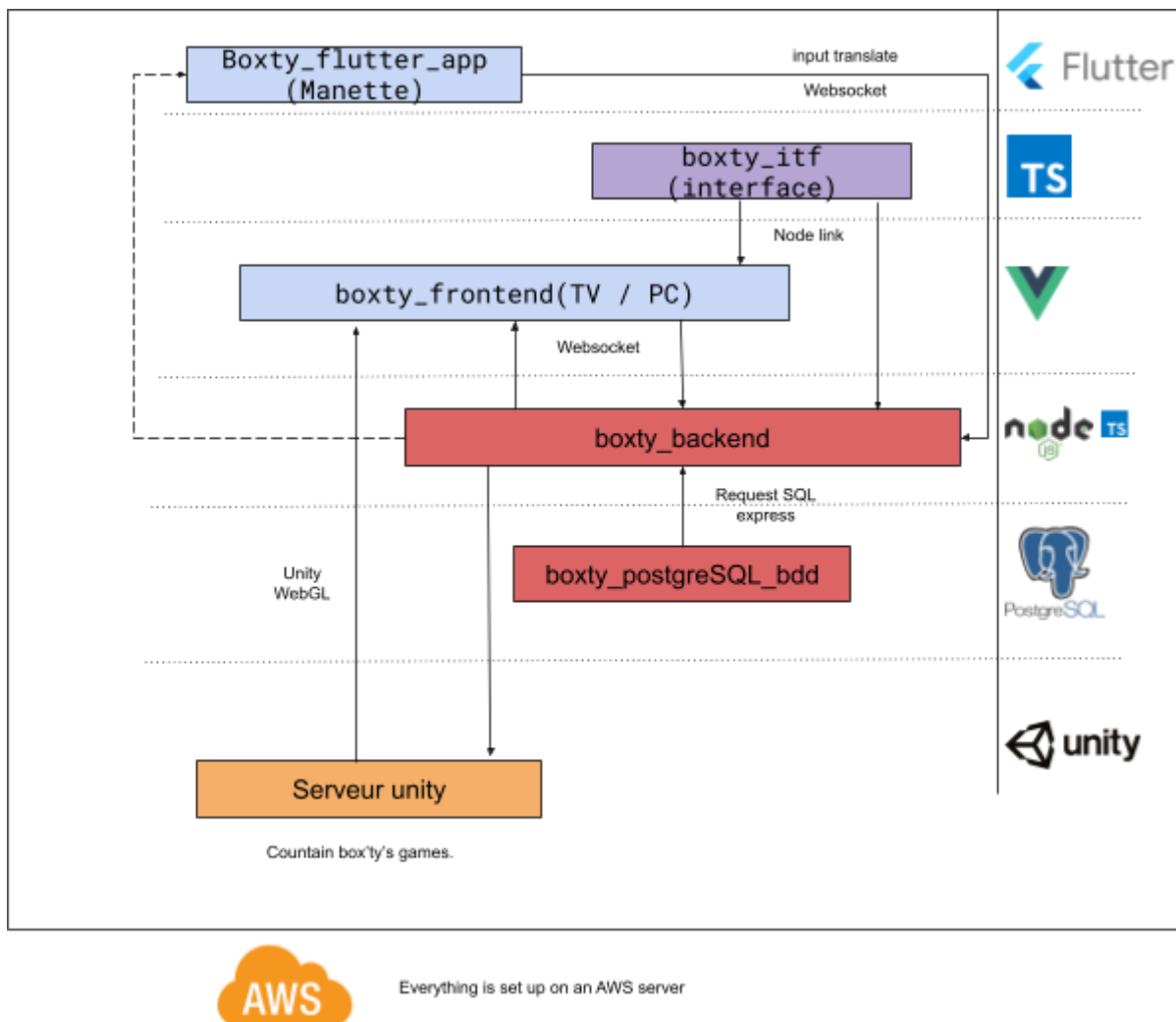


Antonin PULBY | Maxime PAJARD
Timothé TORRES | Kevin FAGOT | Maxime JOULLIN

SOMMAIRE

I / Architecture détaillée de l'application	2
II / Description complète de ses interfaces	4
Maquette 1 : (Page d'accueil)	5
Maquette 2 : (Page de la création du compte)	6
Maquette 3 : (Page de connexion)	7
Maquette 4 : (Page privilège du compte)	9
Maquette 4bis: (Page paramètres user)	10
Maquette 4ter : (Page partie personnalisée)	11
Maquette 4quarter : (Page Abonnement)	12
Maquette 5 : (Page code salon)	13
Maquette 6 : (Page Salon de jeux)	15
Maquette 7 : (Page Détails jeux/ Lancement)	18
Maquette 8 : (Page Session Jeu)	20
Maquette 9 : (Page Réponse Quizz)	21
III / Dictionnaire des données	22
IV / Diagrammes de classes	24
Diagramme de classe : Application Mobile : Flutter.	24
Diagramme de classe : BackEnd : NodeJs - TypeScript.	25
Diagramme de classe : FrontEnd: VueJs.	26
V / Modélisation de l'application	27
Génération d'un code salon :	27
Envoi d'un input depuis Flutter vers le jeu :	28
Choix d'un jeu depuis un ordinateur :	28
Lancement d'un jeu depuis une manette :	29
VI / Argumentation du choix de la méthode de développement	30
Partie Mobile	30
Partie Frontend	31
Partie Backend	31
Partie Jeux	32
Partie Architecture, conteneurisation et infrastructure de travail	33
Déploiement sur serveur externe	34
Méthodes agiles	35

I / Architecture détaillée de l'application



- Les joueurs sont sur une application Flutter pour se connecter et participer à un salon de jeu. L'un des joueurs est alors considéré comme hôte pour contrôler la mise en place, la configuration et le déroulement du jeu.
- Les inputs de Flutter sont émis via WebSocket
- Les WS sont reçues par un Back en TypeScript et Node.js pour traduction et renvoient vers le front
- Le front JS est en Vue.js, ce sera l'interface principale que les joueurs vont regarder.
- La sauvegarde des comptes utilisateurs est stockée sur une Base de données PostgreSQL
- Les requêtes sont faites en Express.js
- Les différents jeux sont développés sous Unity, ils sont intégrés via un Build WebGL au serveur, ils seront téléchargés sur le FrontEnd lors du lancement de la partie.

- Connexion à un écran de SmartTV / Ordinateur
- L'hôte de la partie peut faire la mise en place soit depuis son propre téléphone soit depuis l'ordinateur où sera joué le jeu.
- Chaque utilisateur peut choisir son nom / se connecter sur son téléphone, puis rejoindre avec un code partie affiché sur la TV / écran ses ami(e)s.

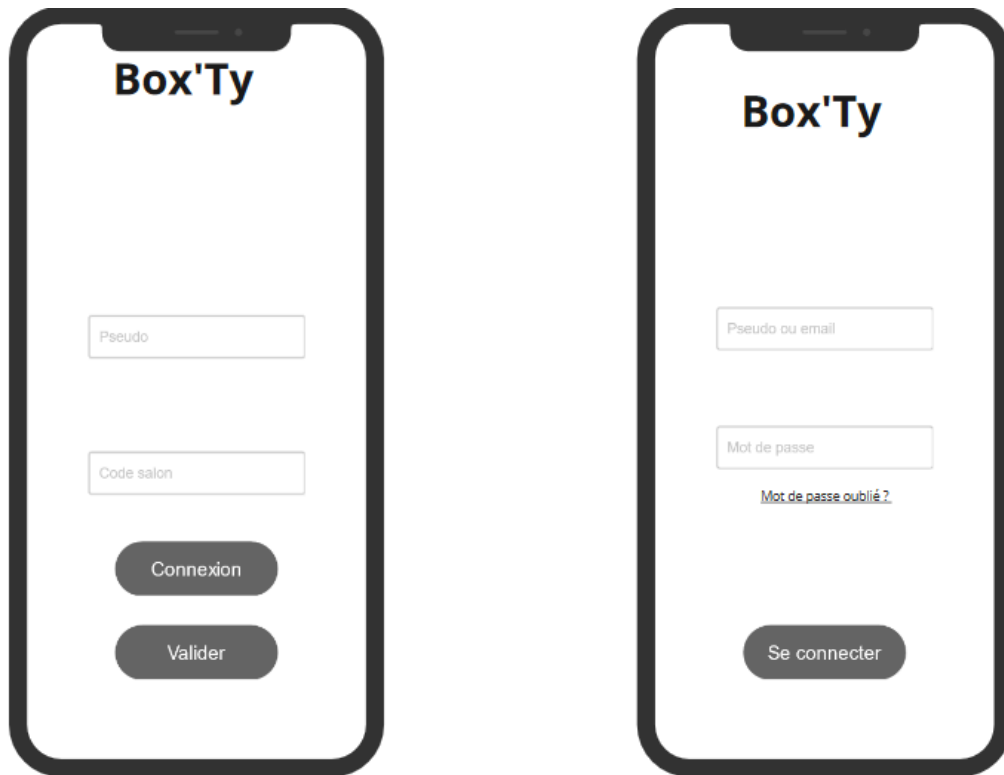


II / Description complète de ses interfaces



Maquette 1 : (Page d'accueil)

- Point de vue de l'hôte (ordinateur) :
l'utilisateur a 3 boutons :
 - Inscription (renvoie sur la maquette 2),
 - Connexion (renvoie sur la maquette 3),
 - Invité (renvoie sur la maquette 4).



- Point de vue joueur/hôte (mobile) :

L'utilisateur peut :

- rejoindre une partie directement en tant qu'invité en saisissant immédiatement son pseudo et le code salon. Renvoie sur la maquette 6
- se connecter, le renvoie sur la maquette 3

Maquette 2 : Page de la création du compte. L'interface est présentée dans un navigateur web. Le header contient le logo 'BOX'TY' et le slogan '"All you need to play is friends"' à gauche, et les liens 'Sign Up | Login' avec un icône utilisateur à droite. Le corps principal de la page est divisé en deux sections : une barre latérale grise à gauche et une zone de formulaire à droite. Le formulaire comprend quatre champs de saisie : 'pseudo', 'Email', 'Mot de passe' (avec un bouton de visibilité à droite), et 'Confirmer le Mot de passe'. En dessous de ces champs se trouve un bouton 'CREER MON COMPTE'.

Maquette 2 : (Page de la création du compte)

- Point de vue hôte (ordinateur)
l'utilisateur doit saisir 4 champs pour créer son compte (pseudo, email, mot de passe, confirmation mot de passe).
Redirection : Maquette 3

The image shows a wireframe for a login page. At the top, there is a header bar with the logo 'BOX'TY' on the left, the tagline '"All you need to play is friends"' below it, and the links 'Sign Up | Login' and a user icon on the right. The main content area contains two input fields: 'Email' and 'Mot de passe'. Below these fields is a button labeled 'CONNEXION'. At the bottom of the form, there is a link that says 'mot de passe oublié ?'.

BOX'TY

"All you need to play is friends"

Sign Up | Login

Email

Mot de passe

CONNEXION

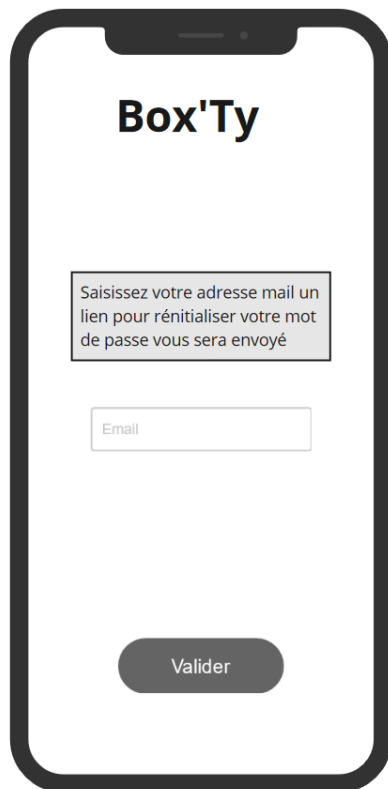
[mot de passe oublié ?](#)

Maquette 3 : (Page de connexion)

- Point de vue hôte (ordinateur)
- Requis : être passé par la maquette 2.

l'utilisateur doit saisir deux champs: son mail et son mot de passe.

Redirection : Maquette 4

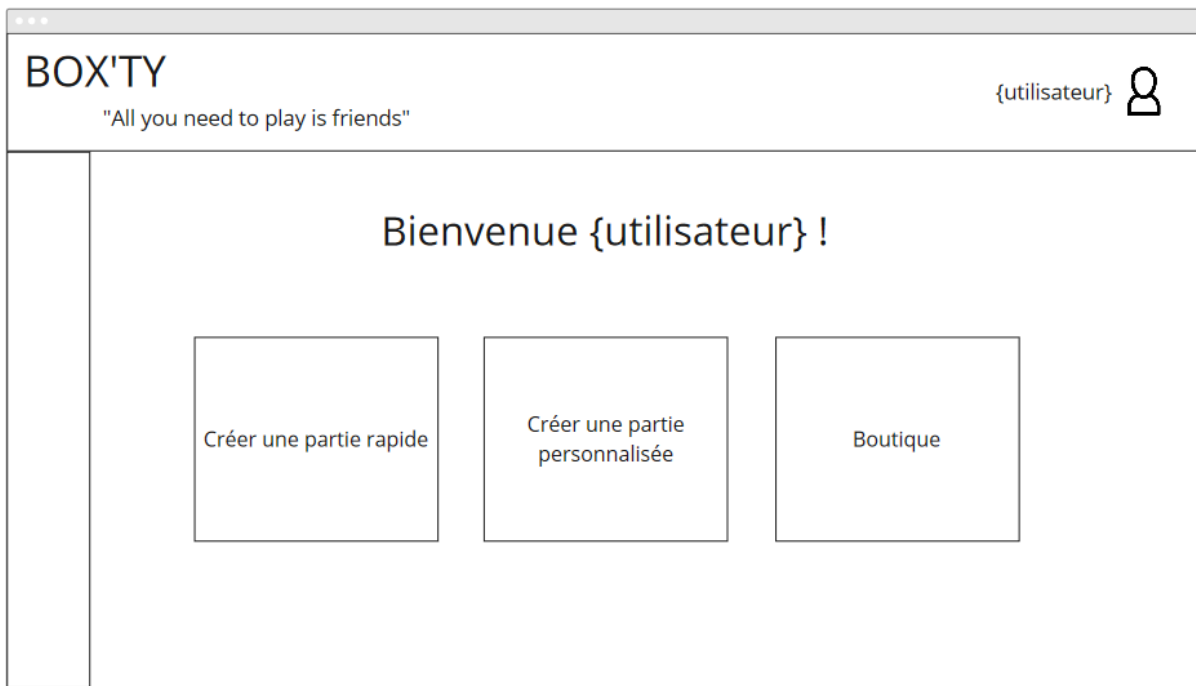


Point de vue hôte/joueur (mobile)

Requis : être passé par la maquette 2.

L'utilisateur doit saisir son pseudo ou son email, il peut aussi faire mot de passe oublié pour régénérer un nouveau mot de passe via son email

Renvoie sur la maquette 5

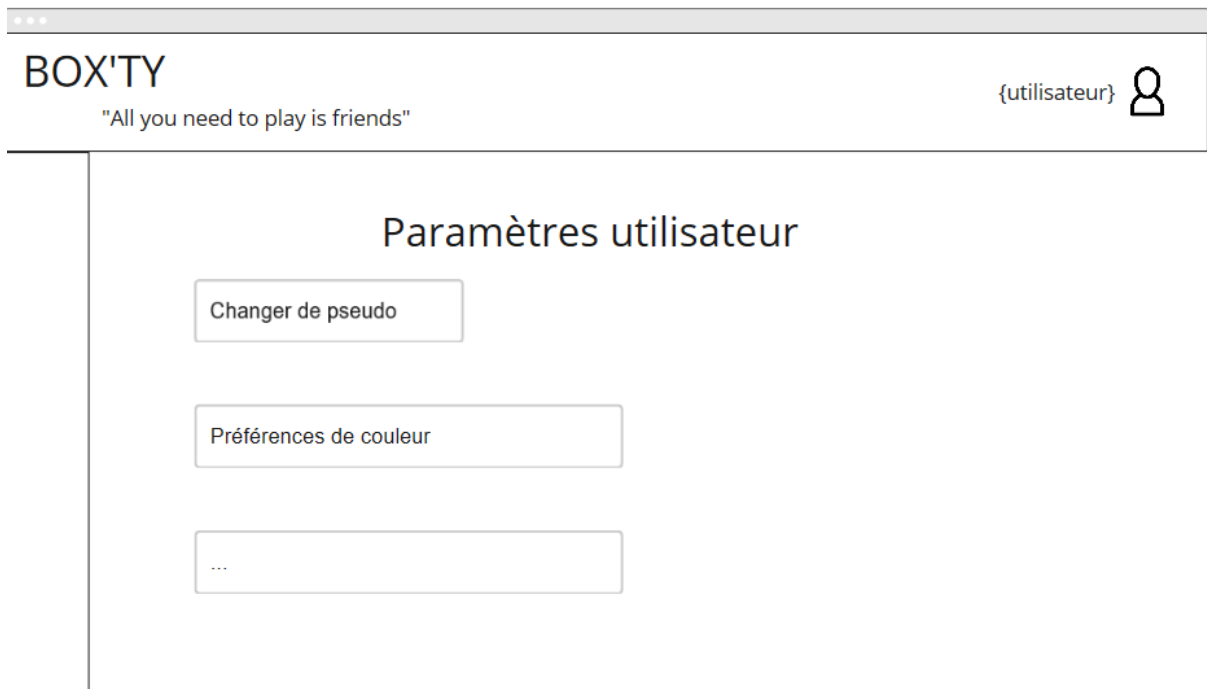


Maquette 4 : (Page privilège du compte)

- Point de vue hôte (ordinateur)

L'utilisateur a accès à 3 menus :

- Créer une partie rapide : renvoie sur la maquette 5
- Créer une partie personnalisée : renvoie sur la maquette 4Q
- Boutique : renvoie sur la maquette 4ter



Maquette 4bis: (Page paramètres user)

- Point de vue hôte (ordinateur)
- Requis : Avoir un compte premium pour certaines fonctionnalités
- Permet de mettre à jour son compte et de s'inscrire aux abonnements.

BOX'TY

"All you need to play is friends"

{utilisateur}

Creer une partie personnalisée

Nombre de mini-jeux

43

Score Maximal

Nombre de mini-jeux

Nombre de mini-jeux

Nombre de mini-jeux

Nombre de mini-jeux

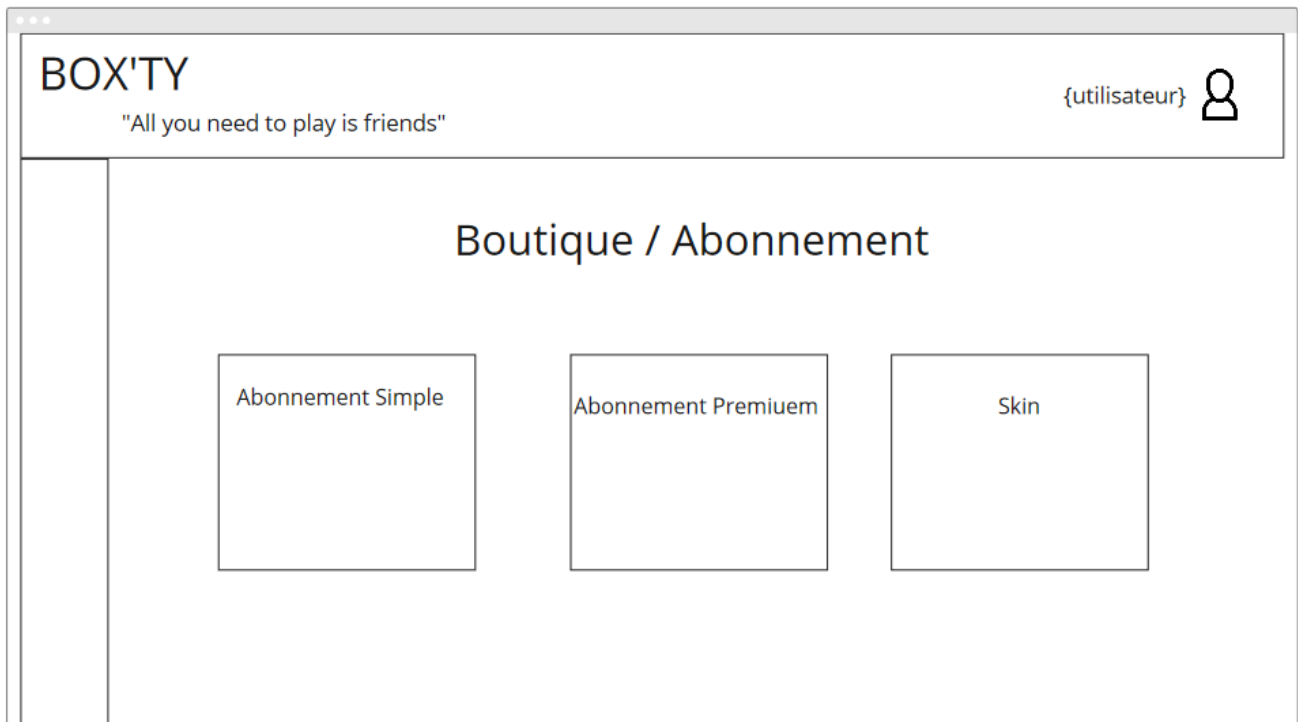
BOUTON

Maquette 4ter : (Page partie personnalisée)

- Point de vue hôte (ordinateur)
- Requis : Avoir un compte premium pour certaines fonctionnalités

L'utilisateur a accès à plusieurs champs pour personnaliser sa partie :

- Nombre de mini-jeux : permet de définir l'ensemble des mini-jeux que comportera la partie
- Score maximal : le score à atteindre avant que le mini-jeux s'arrête
- Bouton de validation : valide les changements appliqués dans les champs, renvoie sur la maquette 5

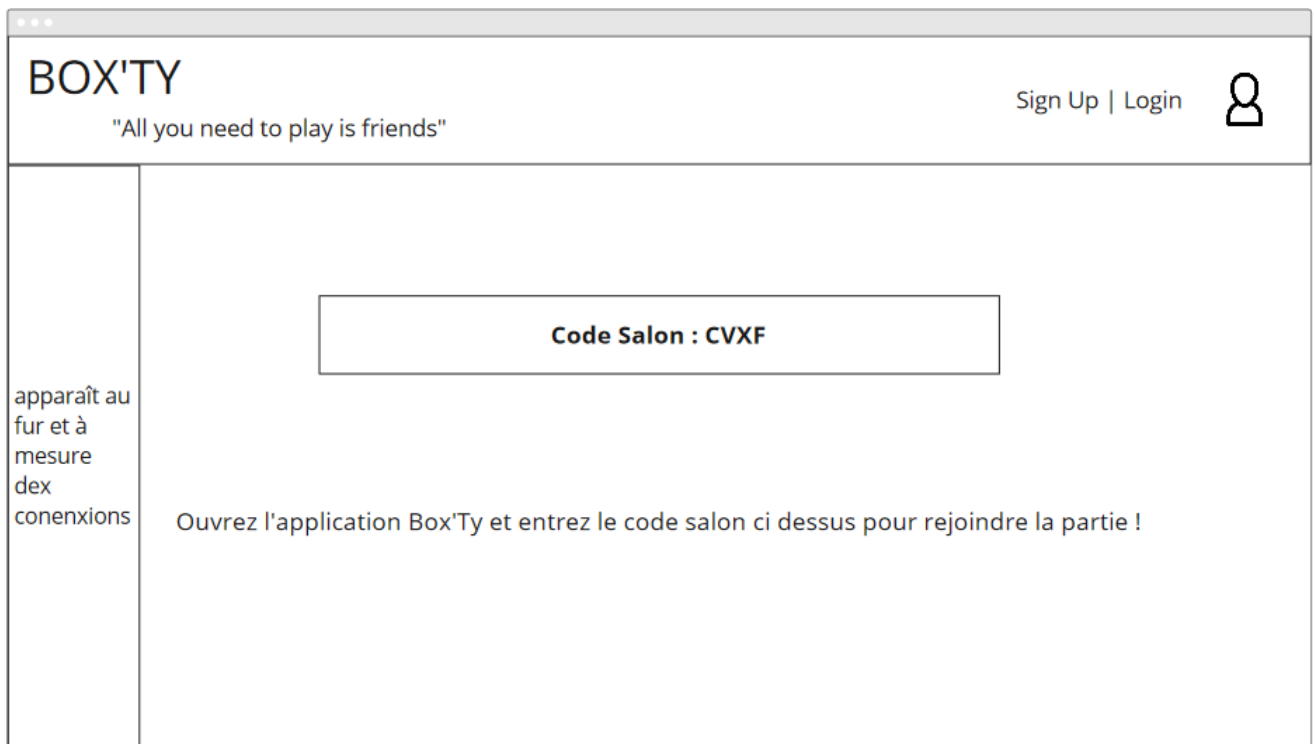


Maquette 4quarter : (Page Abonnement)

- Point de vue hôte (ordinateur)

L'utilisateur a accès à différents boutons :

- Abonnement simple : permet à l'utilisateur de prendre un abonnement pour avoir accès à plus de fonctionnalités.
- Abonnement Premium : permet à l'utilisateur de prendre un abonnement plus cher que l'abonnement simple et qui apporte plus de fonctionnalités.
- Skin : permet à l'utilisateur de personnaliser le thème de sa page.

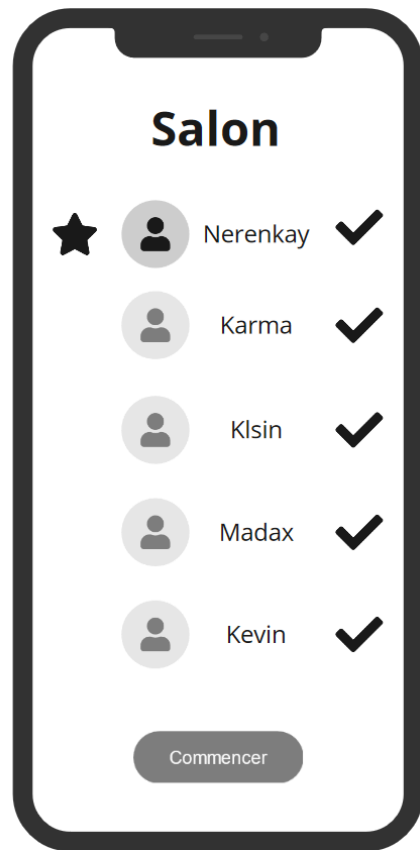
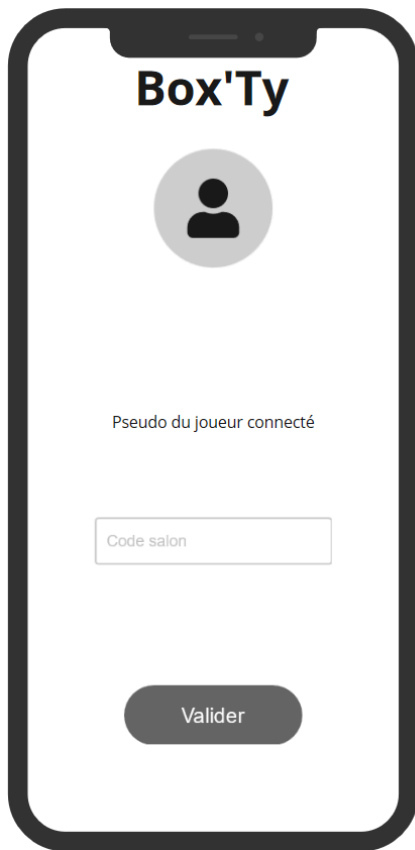


Maquette 5 : (Page code salon)

- Point de vue hôte (ordinateur)
- Requis : Maquette 3

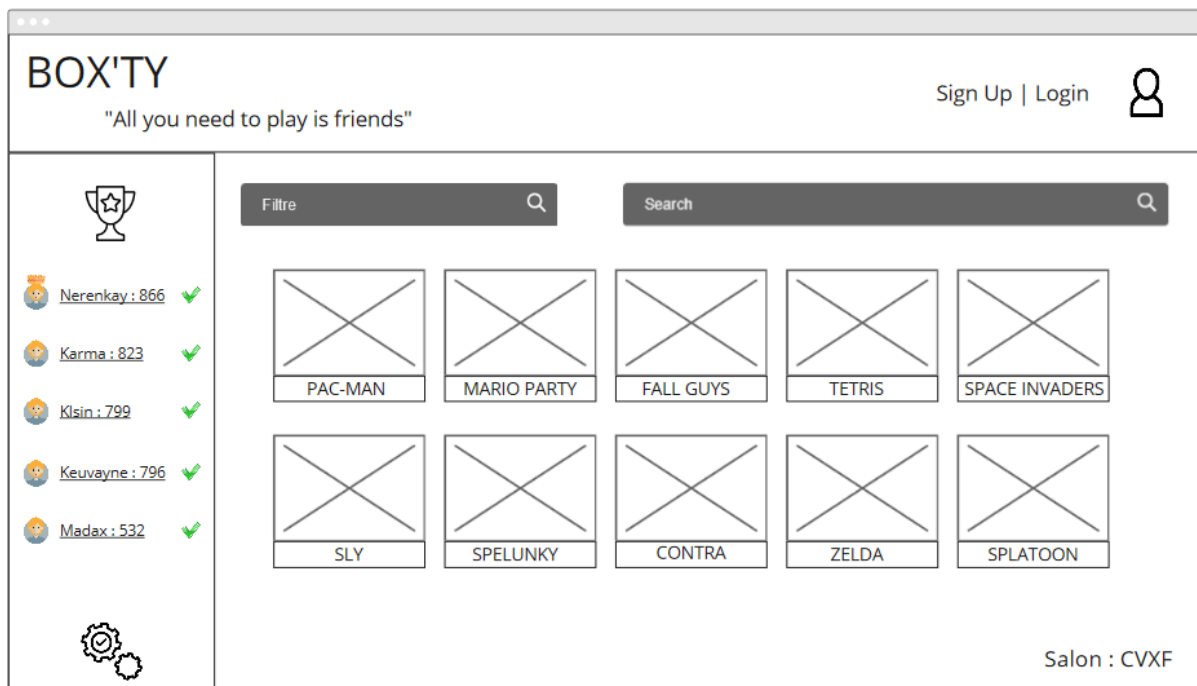
L'utilisateur a accès à une page qui lui génère un code salon au centre et une liste des utilisateurs qui ont rejoint la session sur une fenêtre à gauche.

Redirection : Maquette 6



- Point de vue joueur (mobile)

L'utilisateur peut saisir le code salon pour rejoindre la partie en saisissant ce dernier et son pseudo s'il n'est pas déjà connecté.



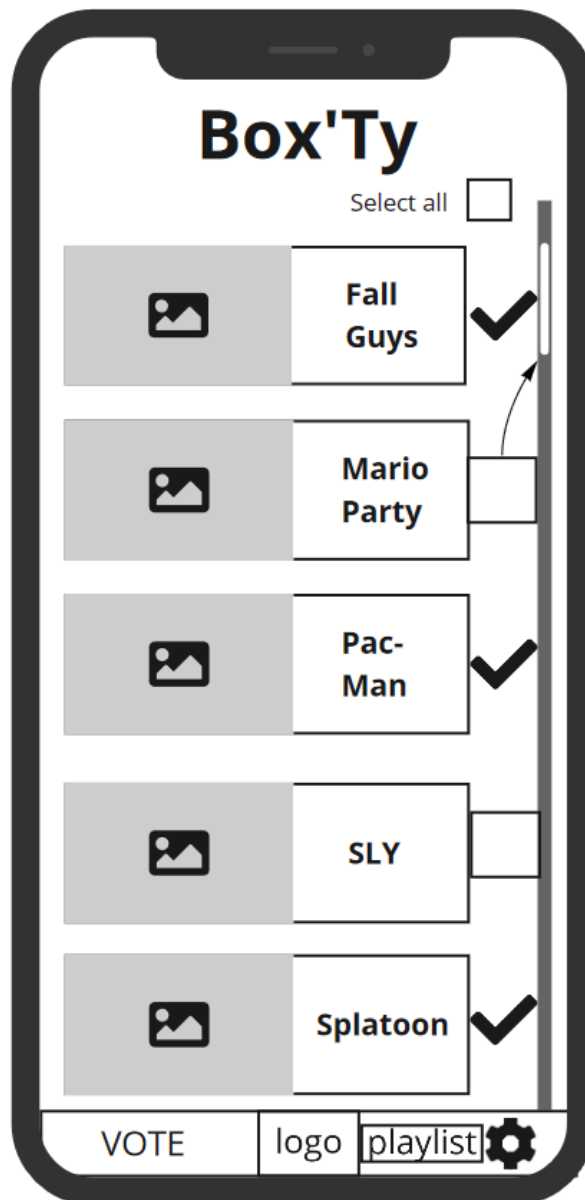
Maquette 6 : (Page Salon de jeux)

- Point de vue de l'hôte (ordinateur)

L'utilisateur a accès à une page regroupant la bibliothèque des jeux mis à disposition par l'application.

Champs sur la page :

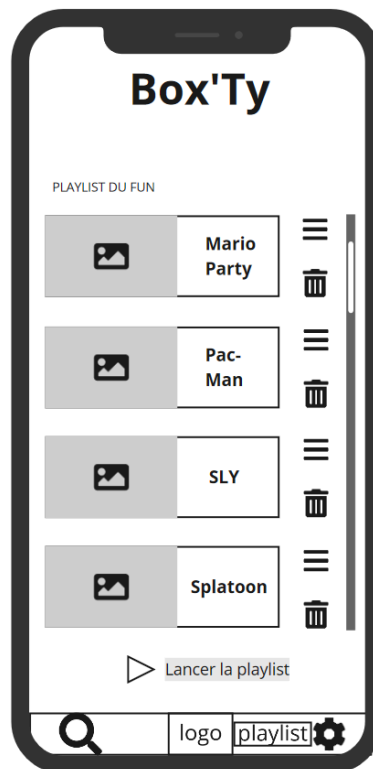
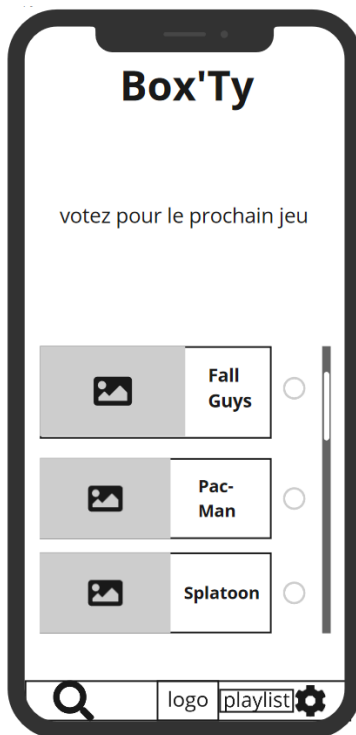
- Filtre : l'utilisateur peut filtrer les jeux afin de les classer par catégories
- Rechercher : L'utilisateur peut saisir des valeurs afin de chercher un jeu précis
- Liste : sur la gauche les utilisateurs retrouvent une liste des membres dans la partie avec leurs scores respectifs.
- Paramètres : accès à une icône paramètres qui permet de modifier les options de la partie, renvoie sur la maquette 4Q
- L'icône d'un jeu : permet de choisir le jeu à lancer, renvoie sur la maquette 7



- Point de vue hôte (mobile)

L'hôte a accès à plusieurs fonctionnalités :

- Bouton playlist : permet de créer une playlist avec une liste des jeux.
- Bouton vote : permet de lancer un vote pour les joueurs présents
- Bouton List playlist : permet de visualiser toutes les playlists déjà créées
- Bouton détails d'une playlist : permet de voir les mini-jeux qui ont été mis dans la playlist



- Point de vue joueur (mobile)

Les utilisateurs peuvent :

- Bouton voter : pour voter pour un mini-jeu
- Bouton playlist détail : permet aux utilisateurs de voir quel mini-jeu est dans la playlist



Maquette 7 : (Page Détails jeux/ Lancement)

- Requis : Maquette 6
- Point de vue hôte (ordinateur)

L'utilisateur peut accéder à la page rassemblant les informations essentielles d'un jeu.

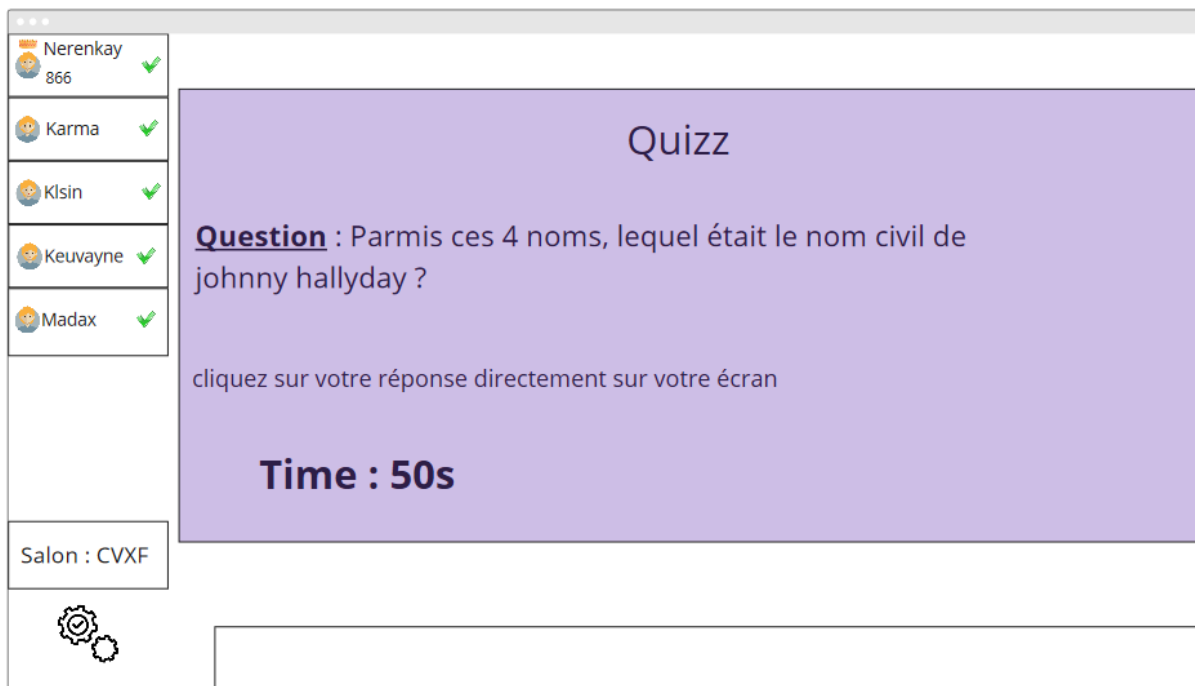
- Titre du jeu
- Les règles du jeu
- Description du jeu
- Liste : sur la gauche les utilisateurs retrouvent une liste des membres dans la partie avec leurs scores respectifs.
- Paramètres : accès à une icône paramètres qui permet de modifier les options de la partie, renvoie sur la maquette 4Q
- Bouton jouer : Lance le jeu et renvoie sur la maquette 8



- Point de vue joueur (mobile)

L'utilisateur peut accéder à une description détaillée du jeu sélectionné.

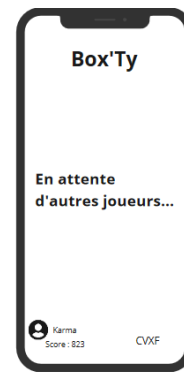
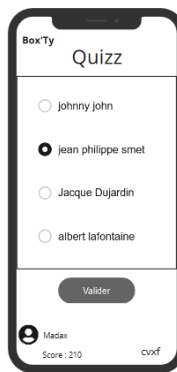
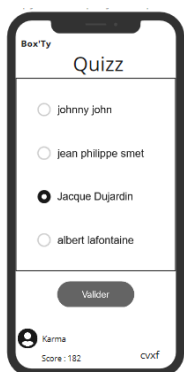
- Titre du jeu + image
- Description du jeu et de ses règles
- Bouton playlist : permet d'ajouter un mini-jeu à une playlist(=regroupe plusieurs mini-jeux)
- Bouton vote : permet au joueur de lancer un vote pour savoir qui est intéressé par ce mini jeu parmi les participants
- Bouton options : permet de définir les paramètres des règles du mini-jeu
- Bouton Paramètre : renvoie sur les paramètres globaux du site



Maquette 8 : (Page Session Jeu)

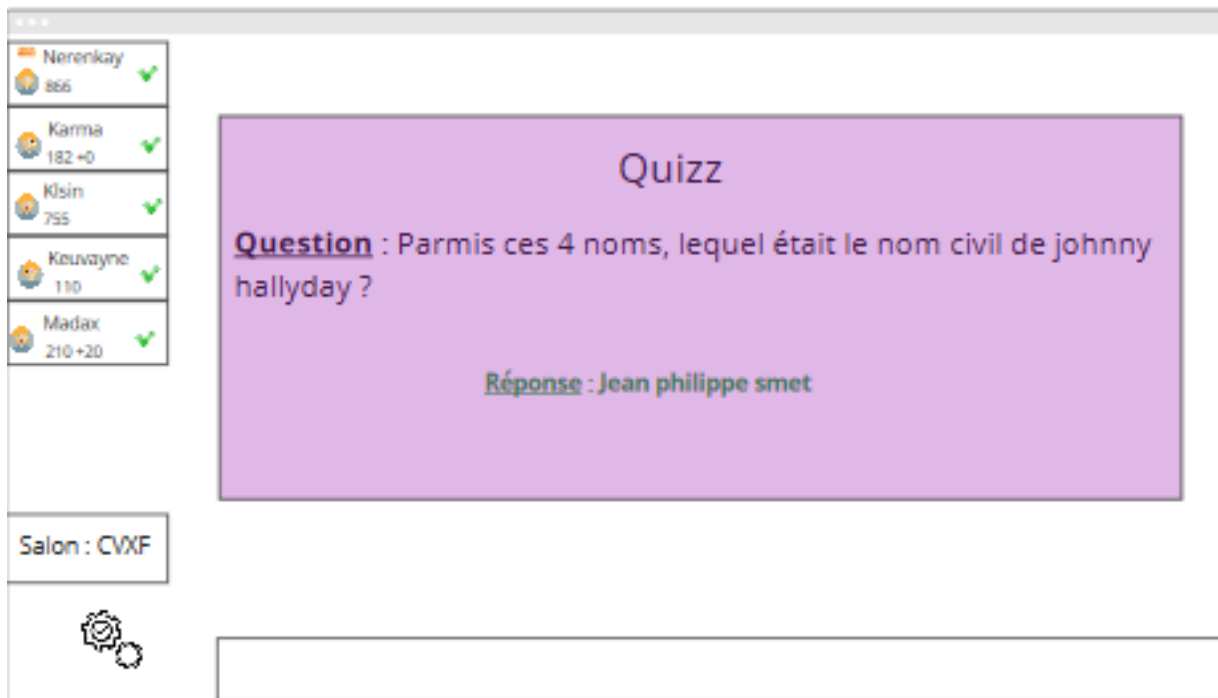
- Point de vue hôte (ordinateur)

L'utilisateur est en train de jouer, la page sert uniquement d'affichage pour le mini-jeu en cours.



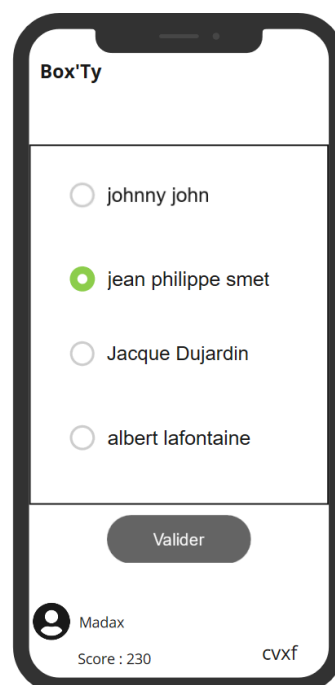
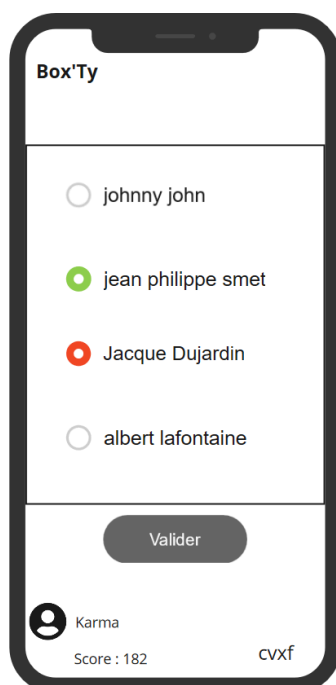
- Point de vue joueur (mobile)

L'utilisateur peut interagir avec le jeu en cours, en appuyant sur les boutons disponibles sur son téléphone (manette). La manette s'adapte à chaque jeu.



Maquette 9 : (Page Réponse Quizz)

- Point de vue hôte (ordinateur)
L'utilisateur peut voir la réponse ou les résultats des mini-jeux en cours



- Point de vue joueur (mobile)
L'utilisateur peut voir son score personnel au mini-jeu en cours

III / Dictionnaire des données

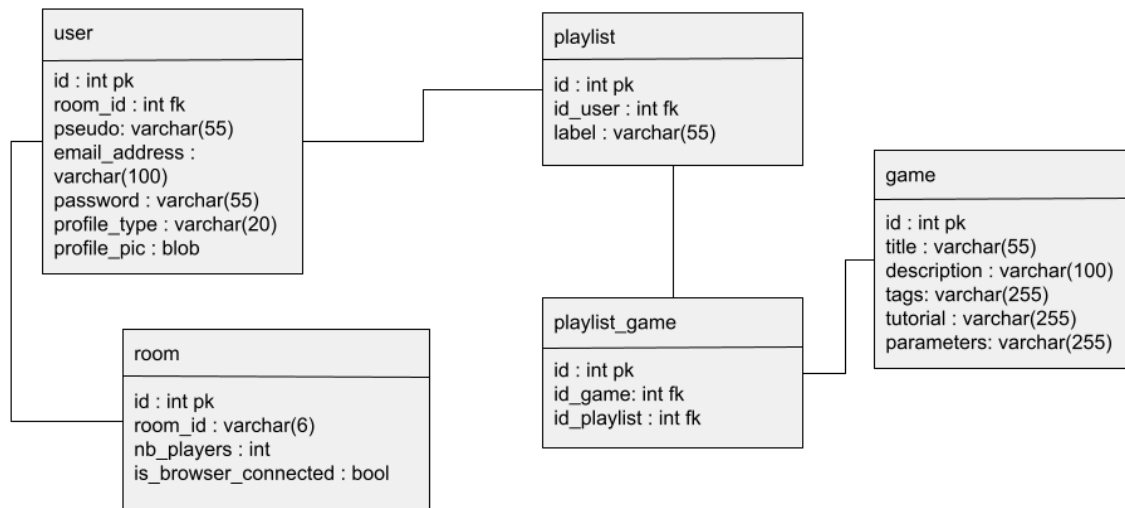


Table User :

- id : id unique pour un user - obligatoire
- pseudo : pseudonyme de l'utilisateur - obligatoire
- email_address : email relié au compte - obligatoire
- password : mot de passe du compte, stocké crypté - obligatoire
- profile_type : type de profil pour différencier les users premiums ou administrateurs - obligatoire
- profile_pic : blob nous permettant de stocker une photo de profil - non obligatoire

Table Playlist :

- id : id unique pour une playlist - obligatoire
- id_user : id de l'utilisateur ayant créé la playlist - obligatoire
- label : nom de la playlist - obligatoire

Table Playlist_game - permet de faire le lien entre une Playlist et un Game:

- id : id unique d'un lien - obligatoire
- id_game : id unique qui référence un jeu (Game) dans la playlist - obligatoire
- id_playlist : id unique de la playlist dans laquelle est contenue le jeu - obligatoire

Table Game :

- id : id unique du jeu - obligatoire
- title : titre du jeu - obligatoire
- description : description du jeu - obligatoire
- tags : pour la recherche, une chaîne de caractères avec différents mots-clés séparés par des espaces - non obligatoire
- tutorial : un texte expliquant le fonctionnement du jeu en quelques lignes (touches, gameplay, but etc...) - obligatoire
- parameters : différents paramètres pouvant être modifiés selon le jeu stocké sous format JSON ({'intitulé du paramètre' : 'paramétrage'}) - non obligatoire

Table Room :

- id : id unique du salon - obligatoire
- room_code : code salon généré par notre module Node.JS lors d'une demande de création de salon (cf diagramme de séquence V / Modélisation de l'application - Génération d'un code salon) - obligatoire
- nb_players : nombre de joueurs/utilisateurs connectés, la page navigateur étant considérée comme un utilisateur dans ce cas. Permet lorsque ce champ arrive à 0 de lancer un timer qui supprimera le salon si personne ne s'y connecte - obligatoire
- is_browser_connected : détermine si le navigateur est connecté à la session. Si aucun navigateur n'est ouvert sur la session, alors un timer se lance afin de fermer et supprimer la session pour le rendre à nouveau disponible. Si un deuxième navigateur essaye de se connecter à un salon, message d'erreur demandant de fermer la première session avant d'en ouvrir une autre (un seul navigateur à la fois) - obligatoire

IV / Diagrammes de classes

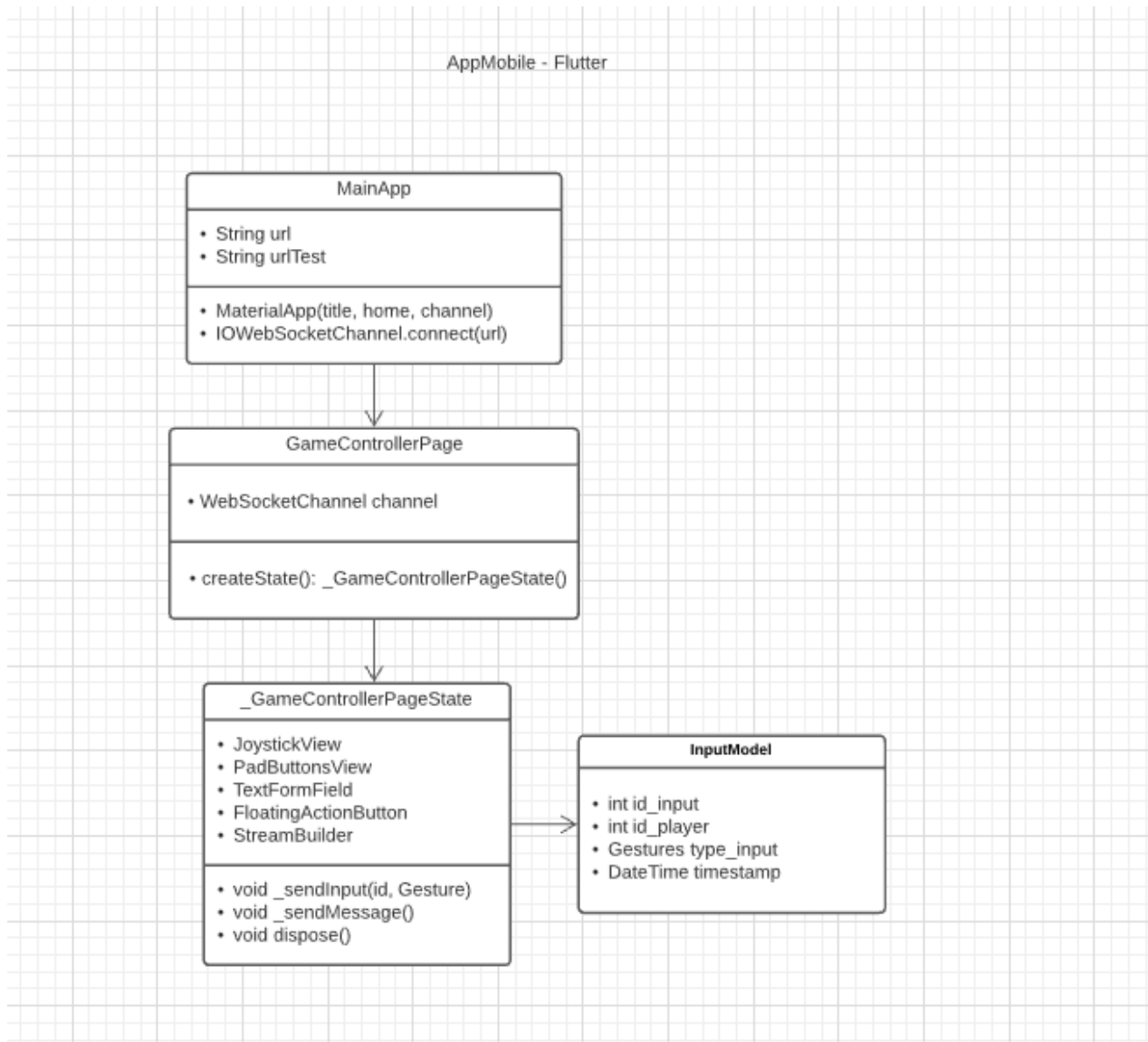


Diagramme de classe : Application Mobile : Flutter. Flutter

L'application mobile est en phase d'évolution, nous avons pour le moment une manette fonctionnelle envoyant des datas d'inputs au backend de l'application.

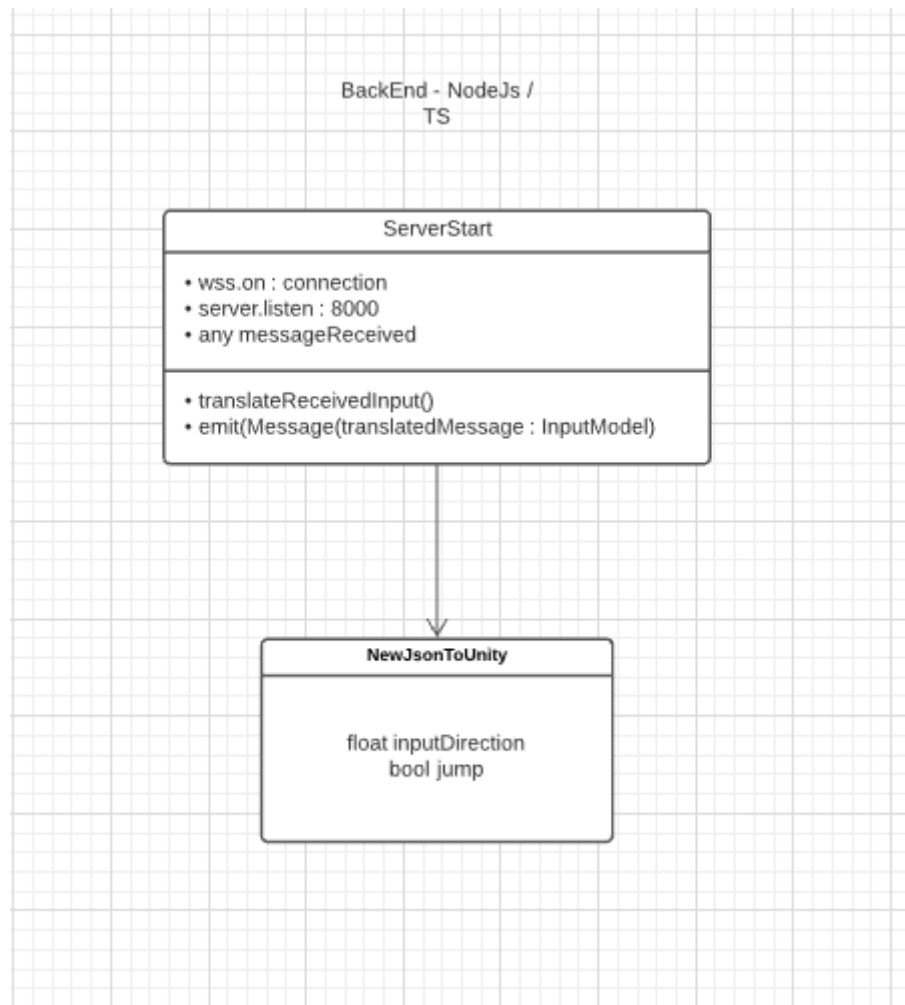


Diagramme de classe : BackEnd : NodeJs - TypeScript.

Le BackEnd se charge de deux choses : il reçoit, par WebSocket, les datas envoyées par l'application en flutter, la manette / téléphone, il reçoit le model sous la forme de l'input model généré par l'application.

Il se charge ensuite de faire une traduction de l'input, pour ensuite l'envoyer au frontend.

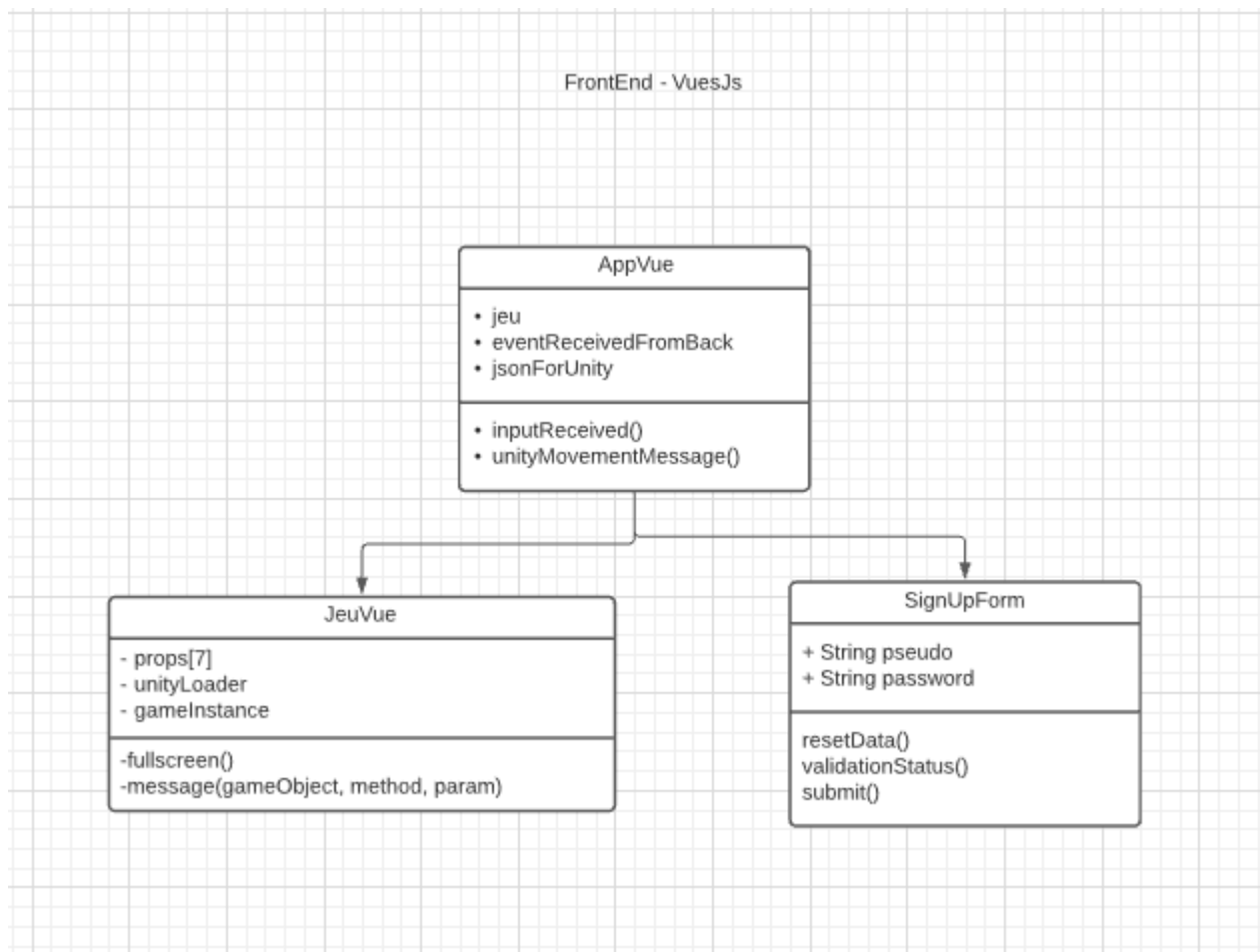


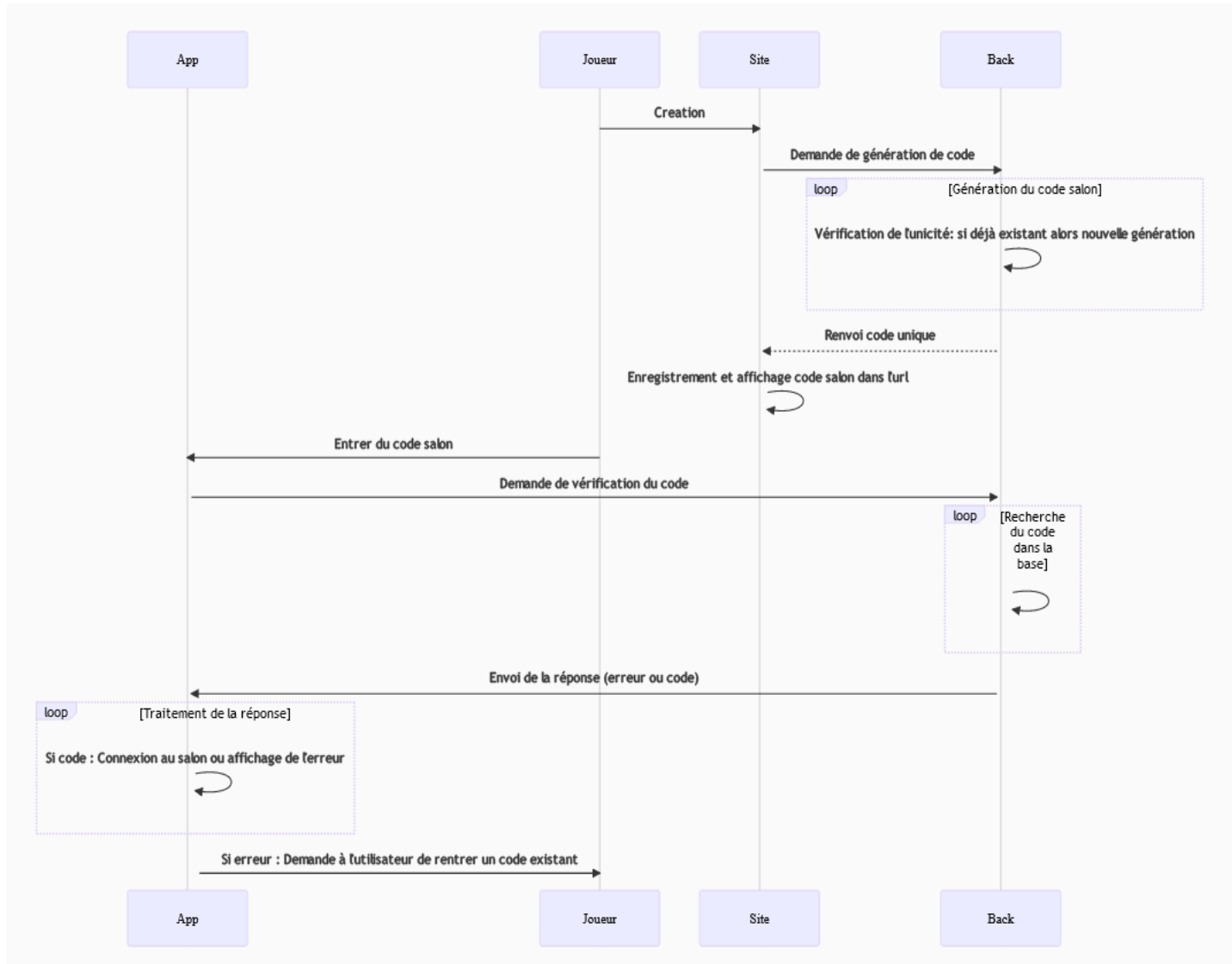
Diagramme de classe : FrontEnd: VuesJs.

Le Frontend reçoit des données directement du back-end sans connaissance de l'application mobile, les transitions de données dans le backend sont renvoyées rapidement car déjà préformatées dans l'application mobile.

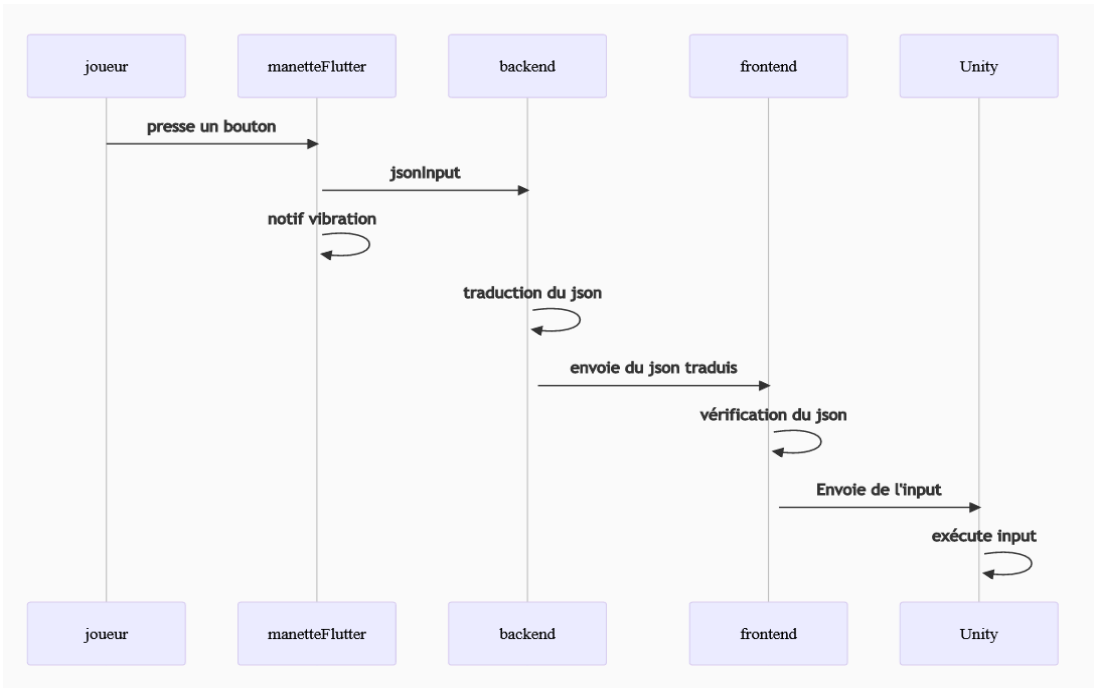
Le front se charge de faire la dernière traduction pour envoyer le bon Input requis par l'application Unity déjà chargé par le front.

V / Modélisation de l'application

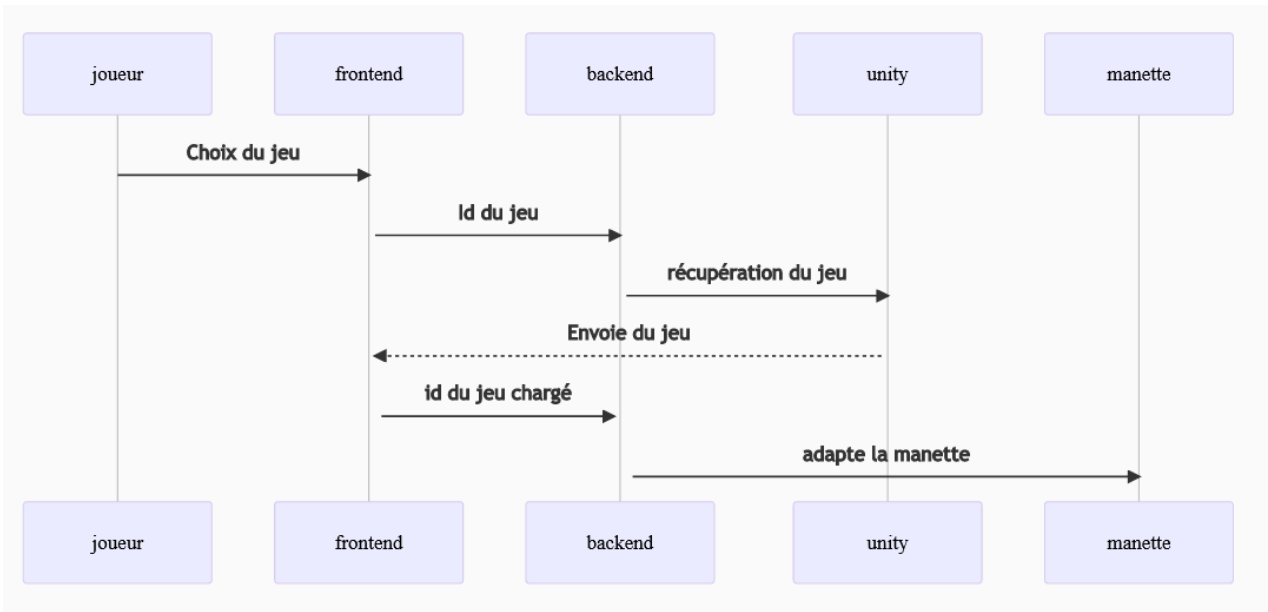
Génération d'un code salon :



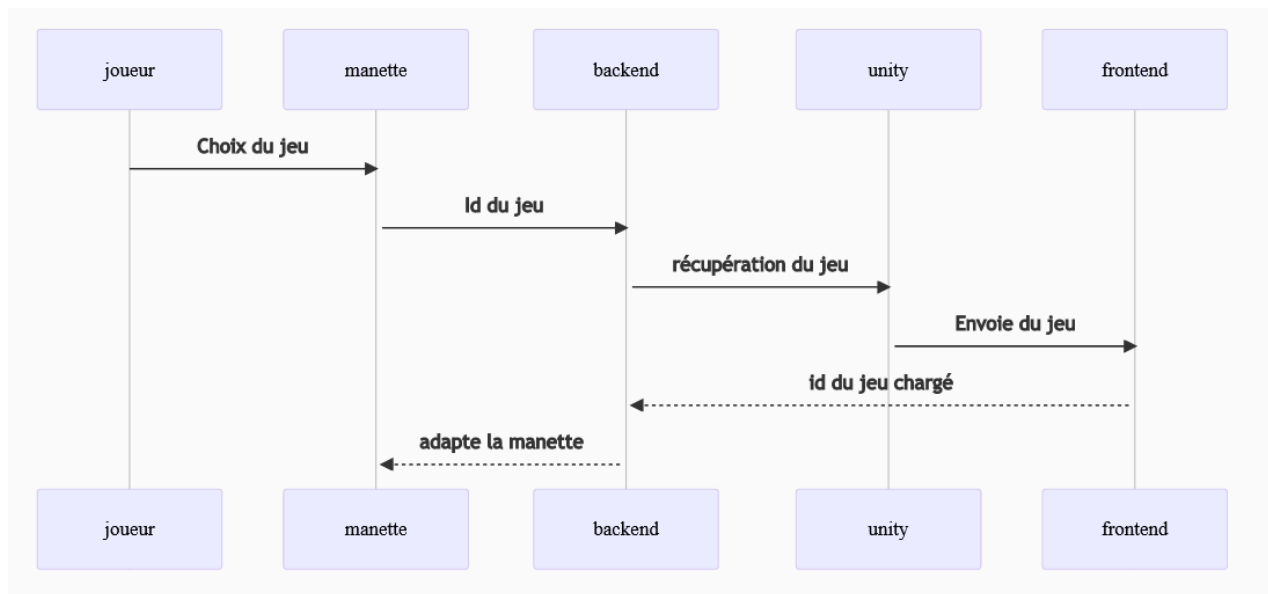
Envoi d'un input depuis Flutter vers le jeu :



Choix d'un jeu depuis un ordinateur :



Lancement d'un jeu depuis une manette :



VI / Argumentation du choix de la méthode de développement

Choix des différentes technologies :

Partie Mobile

Flutter (Dart)  Flutter

- Alternatives :
 - React native : Bonne alternative, déjà quelques connaissances, mais de meilleures arguments pour Flutter
 - Vue Native : Pas de connaissance et une communauté assez peu développée et active mais en concordance avec le frontend en VueJS
- Pourquoi ce choix ? :
 - Une possibilité d'intégrer facilement un projet Unity, la manette étant à la base prévue comme un projet Unity intégré à l'application
 - Recommandé par plusieurs personnes autour du projet
 - Une technologie très intéressante que nous avons envie de découvrir
 - Malgré l'abandon des manettes sous Unity, nous avons gardé Flutter car nous avons déjà commencé à développer l'application et que la technologie nous intéressait encore plus.
 - La création de manette nativement en Flutter n'était pas si complexe de par l'existence de différents plugins nous permettant d'intégrer facilement et rapidement des composants de joysticks et/ou boutons

Partie Frontend

Vue.js



- Alternatives:
 - ReactJS : évoqué pour être en lien avec ReactNative si utilisé. Après abandon de ReactNative, nous avons préféré nous diriger vers VueJS, la technologie nous attirant bien plus
 - AngularJS : évoqué un temps mais vite abandonné car une mise en place plus complexe que ReactJS ou VueJS et plus utilisé pour de lourdes applications complexes
- Pourquoi ce choix ?:
 - Une bonne connaissance du framework par un membre de l'équipe
 - Une grande envie d'apprentissage du framework par toute l'équipe
 - Désir de se mettre à jour avec les frameworks en vogue

Partie Backend

Node.js



- Alternatives :
 - Java : De bonnes connaissances du langage par beaucoup des membres de l'équipe mais pas forcément pour une utilisation Web
 - PHP : De mauvaises expériences avec le PHP et un certain rejet de ce langage par une partie de l'équipe ont vite écarté le langage de notre choix alors que celui-ci aurait pu être un très bon choix
- Pourquoi ce choix ? :
 - Une utilisation de plus en plus fréquente dans les projets web et une grand communauté réactive
 - Quelques connaissances par certains membres de l'équipe pouvant nous aider à apprendre le langage

- Nous avons aussi essayé de garder une certaine cohérence dans les langages : un frontend ET un backend en JavaScript nous permettent d'avoir un apprentissage et une productivité accrue car moins de difficultés à apprendre les subtilités de pleins de langages différents
- Une envie d'approfondir et/ou d'apprendre le langage très prononcée dans l'équipe.

Typescript



- Permet une très bonne adaptation et une évolution importante
- En tant que débutants, le typage permet une propreté accrue du code et une meilleure rigueur dans la conception du code
- Utilisation d'import nous permettant de savoir d'où viennent les imports, ce qui apporte une meilleure lisibilité du code
- Permet un code JS compilé et une meilleure utilisation de la POO
- Nous avons aussi pour but d'intégrer TypeScript à notre partie VueJS dans un futur proche

Partie Jeux

Unity



- Alternatives :
 - JavaScript : Faire les jeux directement en JavaScript aurait été une possibilité mais très vite limitante pour les envies et ambitions que nous avons
 - Unreal Engine : Peu de connaissances par rapport à Unity et nous ne savions pas s'il était possible d'intégrer un jeu Unreal Engine sur navigateur

Pourquoi ce choix ? :

- Une bonne connaissance de la technologie par certains membres de l'équipe
- Une possibilité d'intégration avec Flutter assez simple (même si vite abandonné, cet argument a beaucoup aidé au choix de Flutter comme technologie mobile)
- Une intégration sur navigateur possible par un build WebGL
- Envie de découvrir pour certains et de monter en compétences pour d'autres

Partie Architecture, conteneurisation et infrastructure de travail



- Pourquoi ce choix ? :

Docker nous permet d'avoir un serveur Nginx permettant de rediriger les connexions entrantes et sortantes vers le port 443 et ainsi ouvrir une connexion sécurisée.

Via Docker nous rajouterons aussi les Token JWT directement à notre application pour une meilleure sécurité des informations notamment de login.

De plus, Docker permet un déploiement facile et rapide de tous les projets en même temps sans avoir à demander à chaque développeur de mettre en place un environnement de travail personnel pour le lancement du projet.

Cela nous permet une façon plus simple de déployer le projet si on est amené à travailler sur de nouvelles machines, à faire débiter un nouveau développeur dans le groupe, à mettre à jour et à garder sous main le projet.



- Pourquoi ce choix ? :

Cet outil nous a permis de référencer nos User Stories, de les prioriser et de pouvoir renseigner leur progression afin d'avoir une vision claire du travail accompli, à accomplir ainsi que l'urgence de la tâche. Chaque tâche contient également la ou les personnes qui y ont été assignées et les différents critères d'acceptation. Nous avons également utilisé des tableaux secondaires pour la préparation des divers dossiers.



- Pourquoi ce choix ?

Nous avons décidé de faire un seul gros méta-repository qui nous permet d'y mettre tous les aspects du projet. En compartimentant les projets ainsi que les tâches, nous évitons que deux personnes travaillent sur le même fichier/dossier et limitons les conflits.

Nous avons également mis en place le système de pull request avec des branches pour tous les aspects du projet. Par conséquent, deux développeurs doivent valider la pull request du développeur qui a fait cette dernière avant de pouvoir la merge. L'utilité du méta-repo permet surtout à chaque membre de l'équipe d'avoir une vue totale sur le projet et de pouvoir toucher à toutes ses parties.

Déploiement sur serveur externe

Utilisation de Amazon Web Service



- Alternatives :
 - Machine physique:

L'un des membres du projet aurait pu installer un serveur chez lui mais cela aurait nécessité une machine assez puissante et la possibilité de l'allumer et de la garder en fonctionnement autant que nécessaire pour l'équipe.

De plus, cela aurait nécessité de la configurer entièrement avec un serveur Apache, un ubuntu serveur ou autre: la mise en place aurait été beaucoup plus complexe
 - Heroku:

Nous aurions également pu utiliser cette solution mais dans le cas de Heroku, il ne nous laisse pas complètement la main sur le serveur et quasiment aucune solution ne proposait le temps d'utilisation gratuit
- Pourquoi ce choix ? :
 - Nous avons pris AWS car il nous permet de mettre en place rapidement notre serveur et d'avoir complètement la main dessus, il nous permet également d'éteindre ou de rallumer notre serveur rapidement et offre un long temps d'utilisation gratuit qui réduit le coût du développement, ce qui est un avantage certain pour les phases de test.
 - L'avantage d'avoir un accès total sur le serveur permet aussi, notamment dans notre situation de débutant, de faire des erreurs et de les comprendre.
 - L'exploration de tous les aspects et du fonctionnement d'un serveur distant est essentiel dans l'apprentissage du déploiement d'un projet !

Méthodes agiles

- Mise en place des méthodes agiles

La mise en place de sprint planning en agile avec l'équipe, de l'estimation de la longueur des tâches et de leur priorisation avec le système MOSCOW nous ont permis de nous organiser très rapidement et d'être efficaces en toutes circonstances, grâce à la facilité d'accès aux différentes informations liées au projet.

Afin d'avoir cette rapidité et facilité d'accès, nous avons mis en place de nombreux outils :

- Un serveur Discord sur lequel sont regroupés toutes les avancées, les points importants, les rendez-vous et des espaces de travail distincts
- Un dossier partagé sur Google Drive nous permettant d'y regrouper tous les fichiers importants, comme les présentations, les schémas, les maquettes ou les dossiers et bien plus encore !
- Un backlog complet et détaillé dans un tableur Google Sheet qui répertorie toutes les features de notre projet ainsi que toutes les stories qui le composent

Le tout étant mis à jour au fur et à mesure des différents sprints, rendus et audits.

Merci d'avoir lu